# Experiment No . 10

## AIM :

**To load and display JSON data using fetch() and jQuery $.getJSON(), and to render the results dynamically in a table.**

## SOFTWARE / TOOLS REQUIRED :

- Visual Studio Code (VS Code) or any text/code editor
- Google Chrome (or any modern web browser)
- HTML, CSS and JavaScript
- jQuery library

## THEORY :

1. **JSON Format:**
   JSON (JavaScript Object Notation) is a lightweight data format commonly used to store information in the form of objects and arrays. For example, a JSON file may contain student details such as ID, name, course, and marks.
2. **Fetching Data:**
   When the web page needs this information, the `fetch()` method sends a request to the JSON file. It works asynchronously, so the browser can request the data without stopping the execution of the rest of the webpage. jQuery provides the shorthand `$.getJSON()` for the same purpose.
3. **Parsing the Response:**
   Once the data is received, the response is converted into a JavaScript object using `response.json()` (with `$.getJSON()` the parsing is done automatically).
4. **Dynamic Row Generation:**
   The data is processed using methods such as `forEach()`. For each object in the JSON array, JavaScript creates an HTML `<tr>` element containing `<td>` elements for the individual values and inserts it into the table's `<tbody>` using innerHTML or DOM methods.
5. **Scalability:**
   The table content is generated automatically from the data rather than being written manually in HTML. If the JSON file contains 10 or 100 student records, the same JavaScript code generates the corresponding number of rows.

## A. fetch() vs $.getJSON()

| Feature | fetch() | $.getJSON() |
|---|---|---|
| Origin | Native browser API (ES6) | jQuery library method |
| Returns | Promise | jqXHR (promise-like) object |
| Parsing | response.json() is called manually | JSON is parsed automatically |
| Error handling | catch() (check response.ok for HTTP errors) | fail() callback |
| Extra library | Not required | jQuery must be included |

## CASE STUDY TITLE :

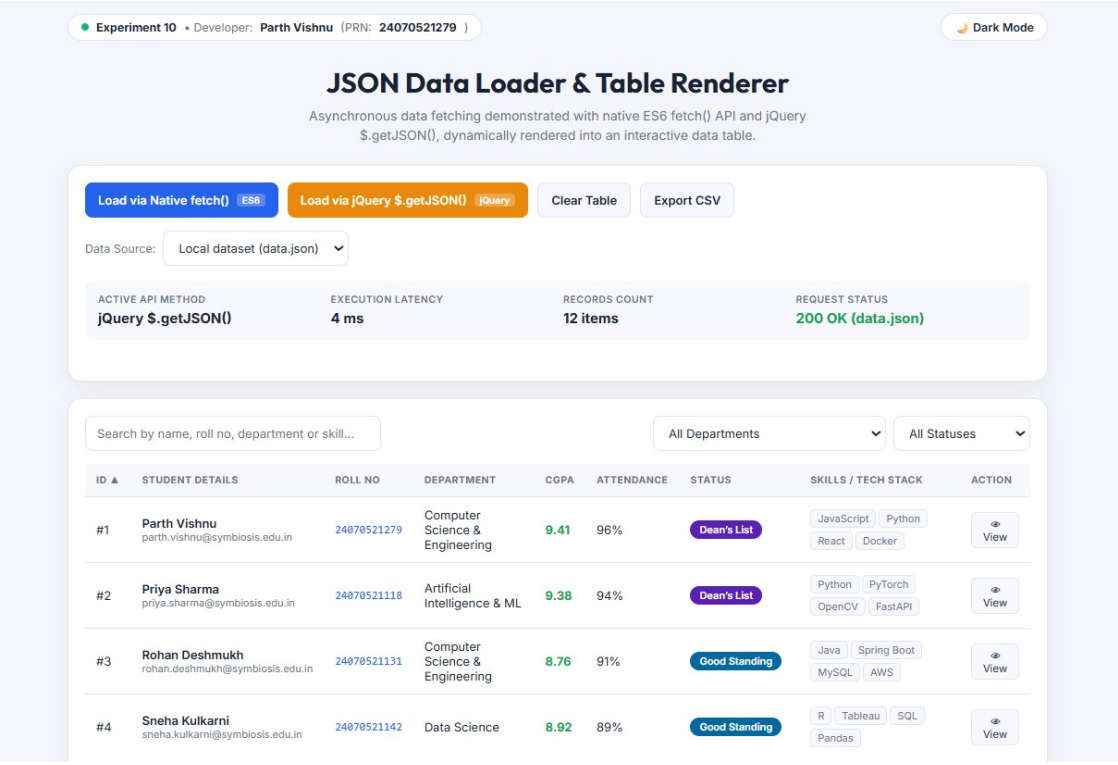
Student Data Loader and Table Renderer using JSON

## AIM OF THE CASE STUDY :

Develop a webpage that loads student records from a JSON file using both `fetch()` and `$.getJSON()` and renders them dynamically in a table. The page should provide search, filter and sort options, show the response time and record count, and allow the table to be cleared or exported as CSV.

Name : Satish Purohit PRN : 24070521268

File path : file:///C:/Users/DELL/exp10/satishpurohit_24070521268/index.html



**OUTPUT :**

On clicking a load button, the program yields:

- **Load via $.getJSON():** the stats panel shows the active method, the execution latency (4 ms), the record count (12 items) and the request status "200 OK (data.json)", and the 12 student records appear as table rows.
- **Load via fetch():** the same records can be loaded with the native fetch() API, and the active method in the stats panel changes accordingly.
- **Table controls:** the table can be searched, filtered by department and status, sorted by any column, cleared, and exported as CSV. The View button shows the full details of a student.

## CONCLUSION :

In conclusion, this experiment demonstrated the loading and display of JSON data using JavaScript `fetch()` and jQuery `$.getJSON()`. The application retrieved student records asynchronously, rendered them dynamically in a table, and provided search, filter and sort functionality along with response latency feedback.

Thus, the experiment demonstrated effective use of asynchronous data fetching and dynamic DOM manipulation in a web application.